Database Management Systems Lab

(CSDC-0232)

Lab File

B. Tech – CSE

4th Semester 2nd Year

ECE & VLSI



Department of Computer Science and Engineering
Dr. B. R. Ambedkar National Institute of Technology, Jalandhar,
Punjab, 144008

(January-June 2026)

Submitted to

Dr. Shobhana Kashyap

Submitted by

Name : **Harsh Joshi**

Roll No : **24104031**

Group : **G2**

INDEX

S. No.	Name of the Practical	Page No.	Date	Signature

PRACTICAL-1

OBJECTIVE

Introduction

- DBMS introduction
- Need for DBMS
- DBMS Architecture
- Database Languages in DBMS
- DBMS Software
- MySQL, Oracle

THEORY

Introduction to DBMS (Database Management System)

A Database Management System (DBMS) is software that is used to store, manage, retrieve, and organize data in a systematic and efficient way. It acts as an interface between the user and the database, ensuring that data is consistently organized, easily accessible, and securely maintained.

Advantages of DBMS over Traditional File-
Based Systems 1. Control of Data Redundancy
In traditional file-based systems, the same data is stored in multiple files for different applications. For example, a student's name and address may be stored separately in admission, examination, and library files. This leads to duplicate copies of the same data, wasting storage space.

Problems caused by redundancy:

- Extra storage cost
- Difficulty in updating data
- Increased chances of inconsistency How DBMS helps:

DBMS stores data in a centralized database where each piece of data is stored only once and shared among applications. This reduces duplication and saves storage while improving data reliability.

2. Data Consistency

Data consistency means that the same data has the same value everywhere in the database. In file systems, if data is updated in one file but not in another, different values of the same data exist. Example:A student's phone number is updated in the admission file but not in the examination file. Result:Inconsistent data leads to incorrect reports and decisions. How DBMS helps:

Since DBMS maintains a single copy of data, any update made to the database is reflected everywhere automatically, ensuring consistent data.

3. Data Integrity

Data integrity refers to the accuracy and correctness of data stored in the database.
In file systems, there are no proper rules to ensure valid data entry. Incorrect or incomplete data can easily be stored.

Examples of integrity issues:

- Entering negative marks for a student
- Inserting a record without a unique ID How DBMS helps:

DBMS enforces integrity constraints such as:

- Primary key (unique values)
- Foreign key (valid references)
- Domain constraints (valid data types and ranges)

These rules prevent invalid data from entering the database.

4. Data Security

Data security is a major concern, especially when sensitive information is involved. File-based systems provide minimal security. Anyone with file access can view or modify data.
Risks include:

- Unauthorized access
- Data modification or deletion
- Data theft

How DBMS helps:

DBMS provides strong security mechanisms:

- User authentication (username and password)
- Authorization (role-based access)
- Access control (read, write, update permissions)

Only authorized users can access specific data, ensuring confidentiality.

5. Data Sharing

Organizations require multiple users to access the same data at the same time. In file systems, sharing data among users is difficult and unsafe. Problems:

- Data conflicts
- No control over simultaneous access How DBMS helps:

DBMS allows controlled data sharing among multiple users. It ensures that:

- Users can work simultaneously
- Data remains consistent and accurate

This is essential in systems like banking, airline reservations, and online shopping.
6. Backup and Recovery

System failures such as power loss, hardware crashes, or software errors can result in data loss. File systems do not have automatic recovery mechanisms.
Consequences:

- Permanent data loss
- Business interruption How DBMS helps:

DBMS provides:

- Automatic backups

Transaction logs
Recovery mechanisms

If a failure occurs, the DBMS can restore the database to its previous consistent state. 7. Data Independence

Data independence means that changes in database structure should not affect application programs. In file systems, any change in file format requires changes in application code. Types of data independence:

- Physical data independence:Changes in storage methods do not affect applications. • Logical data independence:Changes in logical structure do not affect user views. How DBMS helps:

DBMS separates data from application programs, allowing flexibility and easier maintenance. 8. Efficient Data Access

In file systems, searching for data requires reading entire files, which is slow and inefficient. Problems:

- Time-consuming searches
- Poor performance with large data How DBMS helps:

DBMS uses:

- Indexing
- Query optimization
- Structured query language (SQL)

This allows fast and efficient retrieval of data even from large databases.

9. Concurrency Control

Concurrency occurs when multiple users access the database simultaneously. Without proper control, this can lead to:

- Lost updates
- Dirty reads
- Inconsistent data

Example:Two users updating the same bank account at the same time.

How DBMS helps:

DBMS uses locking, timestamps, and transaction management to ensure that: • Multiple users can work safely

- Data remains correct and consistent

DBMS Architecture with Time Complexity

We'll discuss three levels of DBMS architecture and analyze how operations like search, insert, update, delete happen, and what affects time complexity.

1. External Level (View Level)Role

User interacts with customized views of the database.

Example: A student sees only their marks; a teacher sees all students' marks. Time Complexity

At this level, no actual data storage is handled.

Complexity depends on the queries executed at lower levels (conceptual/internal). Overhead is mostly O(1) to fetch the view definition.
Example: SELECT * FROM Student WHERE ID=123

The DBMS just maps it to the conceptual schema; actual search happens below. Observation: External level adds minimal overhead, constant time mapping.

2. Conceptual Level (Logical Level)Role

- Defines the structure of the database and constraints.
- Handles integrity rules and relationships. Operations

- Search: Check constraints, foreign keys, primary keys.
- Insert/Update/Delete: Must validate rules (e.g., uniqueness, referential integrity). Time Complexity

Depends on size of data (n = number of records in table) Without indexes:

Search: O(n) – linear scan

Insert: O(1) for adding to table, O(n) if uniqueness check needed

- Update/Delete: O(n) to find records

With indexes (e.g., B-trees, hash tables):

- Search: O(log n) – binary tree lookup
- Insert/Delete: O(log n) – tree update
- Constraint checks: O(log n) with index on keys

3. Internal Level (Physical Level)Role

- Deals with how data is stored physically on disk.
- Uses indexes, blocks, clustering for efficient storage. Operations

- Search: Disk read + index traversal
- Insert/Delete/Update: Modify disk blocks and indexes
- Concurrency control: Locking and transaction management

Database Languages in DBMS

A database language is a set of commands used to define, manipulate, and control data in a database. DBMS supports multiple types of languages:

A. Data Definition Language (DDL)

Purpose: To define the structure of the database, such as tables, schemas, and constraints. Common Commands:

- CREATE → to create a table or database
- ALTER → to modify table structure
- DROP → to delete a table or database

Example:

```
CREATE TABLE Students (
 Student_ID INT PRIMARY KEY,
 Name VARCHAR(50),
 Age INT
);
```

Role in DBMS: Ensures data integrity and schema definition at the conceptual level.
B. Data Manipulation Language (DML)

Purpose: To insert, update, delete, and retrieve data from the database. Common Commands:

- INSERT → add new records
- UPDATE → modify existing records
- DELETE → remove records
- SELECT → retrieve data Example:

INSERT INTO Students VALUES (101, 'Alice', 20);

SELECT * FROM Students WHERE Age > 18;

Role in DBMS: Used by users and applications to interact with data.

C. Data Control Language (DCL)

Purpose: To control access and permissions to the database.

Common Commands:

- GRANT → provide permissions
- REVOKE → remove permissions Example:

GRANT SELECT, INSERT ON Students TO Teacher;

REVOKE INSERT ON Students FROM Student;

Role in DBMS: Provides security and access control.

D. Transaction Control Language (TCL)

Purpose: To manage transactions and ensure ACID properties (Atomicity, Consistency, Isolation, Durability).

Common Commands:

- COMMIT → save changes permanently
  ROLLBACK → undo changes
  SAVEPOINT → mark a point within a transaction

Example:

BEGIN;

UPDATE Students SET Age = 21 WHERE Student_ID = 101;

COMMIT;

Role in DBMS: Ensures reliability and concurrency control.

Relational DBMS (RDBMS)

Definition:A DBMS that stores data in tables (relations), where each table is related to others through keys.

Structure:Rows = records, Columns = attributes. Relationships are defined via primary keys and foreign keys.

Examples:MySQL, Oracle, SQL Server, PostgreSQL Key

Features:

- Uses SQL for queries
- Ensures ACID compliance
- Supports indexes, joins, and constraints Use Cases:
- Banking systems
- University student databases
- Online retail applications

Example:A university uses MySQL to store students, courses, and marks in tables with relationships between them.

2. Object-Oriented DBMS (OODBMS)

Definition:Stores data as objects, similar to objects in programming languages like Java or C++. Structure:Objects store data and methods together.
Key Features:

- Good for complex data types (multimedia, images, CAD models)
- Supports inheritance, encapsulation, and polymorphism

Examples:db4o, ObjectDB, Versant Use

Cases:

- CAD (Computer-Aided Design) applications
- Multimedia databases
- Scientific simulations

Example:A CAD application storing 3D object designs along with operations like rotate or scale uses an OODBMS.

3. NoSQL DBMS

Definition:A non-relational DBMS that stores unstructured or semi-structured data like JSON, XML, or key-value pairs.

Structure:Flexible, schema-less design Key

Features:

- High scalability and availability

- Handles large volumes of big data
- Optimized for fast read/write operations

Examples:MongoDB, Cassandra, CouchDB Use Cases:

- Social media feeds
- Real-time analytics
- IoT applications

Example:Facebook uses a NoSQL database to store unstructured user activity and posts for millions of users efficiently. countries while maintaining consistency and real-time access.

MySQL

Overview:

MySQL is an open-source relational database management system (RDBMS). Developed by MySQL AB, now maintained by Oracle Corporation.

Uses Structured Query Language (SQL) for database operations.

Designed for speed, reliability, and ease of use, especially in web applications. Key Features of MySQL

- Open-Source & Free• Community version is free to use and modify.
- Relational Database Model• Stores data in tables with rows and columns.
- Supports primary keys, foreign keys, and constraints.
- ACID Compliance• Transactions are Atomic, Consistent, Isolated, Durable, ensuring reliability. • Multi-User Access & Concurrency• Allows multiple users to access and update data simultaneously.
- Indexing & Query Optimization• Supports B-Tree, Hash, and Full-text indexes for fast data retrieval.
- Storage Engines• MySQL supports InnoDB (transactional) and MyISAM (nontransactional) storage engines.
- Cross-Platform Support• Works on Windows, Linux, macOS, and Unix.
- Replication & Backup• Supports master-slave replication and automated backups.

Advantages of MySQL

- Free and open-source (community edition)

- Lightweight and fast, ideal for web apps
- Easy to integrate with programming languages (PHP, Python, Java)
- Large community support Use Cases of MySQL

- Websites like WordPress, Facebook, YouTube
- E-commerce platforms like Magento
- Small to medium-scale enterprise applications

2. Oracle Database

Overview:

Oracle Database is a commercial, enterprise-level RDBMS.

Known for high scalability, reliability, and security.

Supports SQL and PL/SQL (procedural extension of SQL).

Key Features of Oracle

- High Scalability• Can handle large amounts of data and thousands of concurrent users. • Data Security• Advanced security features like authentication, encryption, and role-based access. • ACID Compliance & Transaction Management• Ensures reliable multi-user transactions.
- Data Warehousing & Analytics• Supports complex queries, OLAP, and reporting for decision-making.

- Backup, Recovery & High Availability• Supports clustering, replication, and automatic failover. • Advanced SQL Support• PL/SQL allows procedural programming inside the database. • Distributed Database Support• Can handle data across multiple nodes in enterprise environments. Advantages of Oracle

- Highly reliable and scalable for large enterprises
- Robust security and data protection
- Handles complex queries and large datasets efficiently
- Provides enterprise-level features like partitioning, clustering, and replication Use Cases of Oracle

- Banks and financial institutions (transaction management)
- Government and defense applications
- Large enterprise ERP systems (SAP, HR systems)
- Airline reservation syste

PRACTICAL-2

OBJECTIVE

To create and modify database tables using DDL commands and to manipulate data using DML commands.

SOFTWARE / TOOLS USED

- MySQL
- MySQL Workbench

THEORY

DDL (Data Definition Language): Commands used to define and modify the structure of database objects such as tables. Examples: CREATE, ALTER, DROP, TRUNCATE.

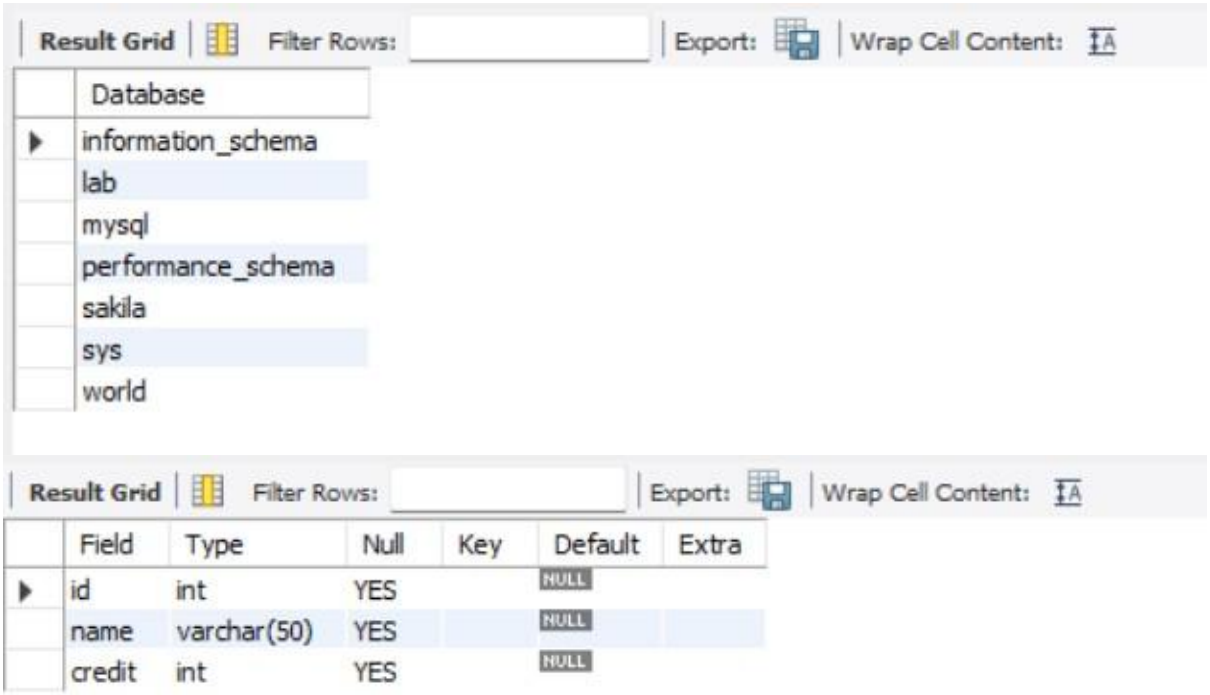
DML (Data Manipulation Language): Commands used to manipulate data stored in tables. Examples: INSERT, UPDATE, DELETE, SELECT.

SQL COMMANDS / PROGRAM

1. Database and Table Initialization

```
sql
show databases;
sql
create database harsh31; use
harsh31;
create table h31(id int, name varchar(50), credit int); describe
h31;
```

OUPUT



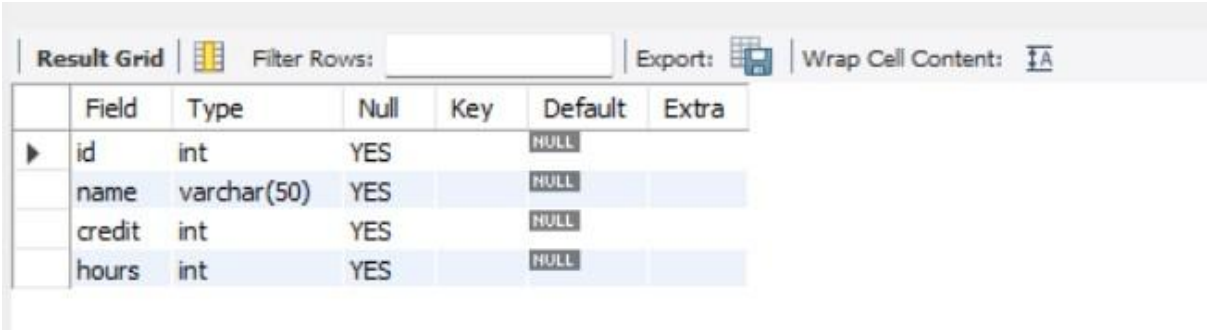
Database
information_schema
lab
mysql
performance_schema
sakila
sys
world

Field	Type	Null	Key	Default	Extra
id	int	YES		NULL	
name	varchar(50)	YES		NULL	
credit	int	YES		NULL	

2. Altering Table Structure

```
sql
alter table h31 add hours int; describe
h31;
```

OUPUT



Field	Type	Null	Key	Default	Extra
id	int	YES		NULL	
name	varchar(50)	YES		NULL	
credit	int	YES		NULL	
hours	int	YES		NULL	

3. Data Insertion

```
sql
insert into h31(id, name, credit, hours) values (1, 'aic',3,3), (2, 'psp', 4,4); select
* from h31;
```

OUPUT



Result Grid | Filter Rows:

	id	name	credit	hours
▶	1	aic	3	3
	2	psp	4	4

4. Operations on tbl31 (Conditional Deletion)

```
sql
use dlt31;
create table tbl31(id int, age int);
insert into tbl31(id, age) values (1,18), (2,49), (3,16), (4,58), (5,19), (6,28);
select * from tbl31;
sql
DELETE FROM tbl31 WHERE age > 28;
SET SQL_SAFE_UPDATES = 0; select *
from tbl31;
```

OUPUT



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	id	age
▶	1	18
	2	49
	3	16
	4	58
	5	19
	6	28

Result Grid | Filter Rows:

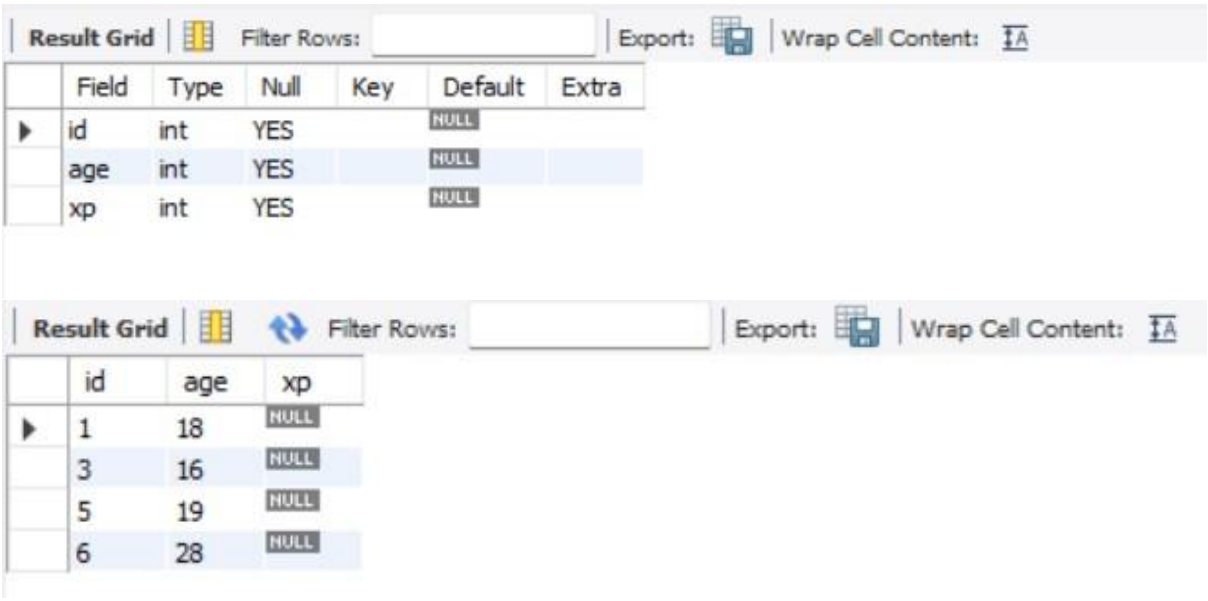
	id	age
▶	1	18
	3	16
	5	19
	6	28

5. Column Addition and Table Inspection

```
sql
alter table tbl31 add xp int;
```

```
describe tbl31; select
* from tbl31; sql
show databases;
use harsh31; show
tables;
select * from h31;
```

OUPUT



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	int	YES		NULL	
age	int	YES		NULL	
xp	int	YES		NULL	

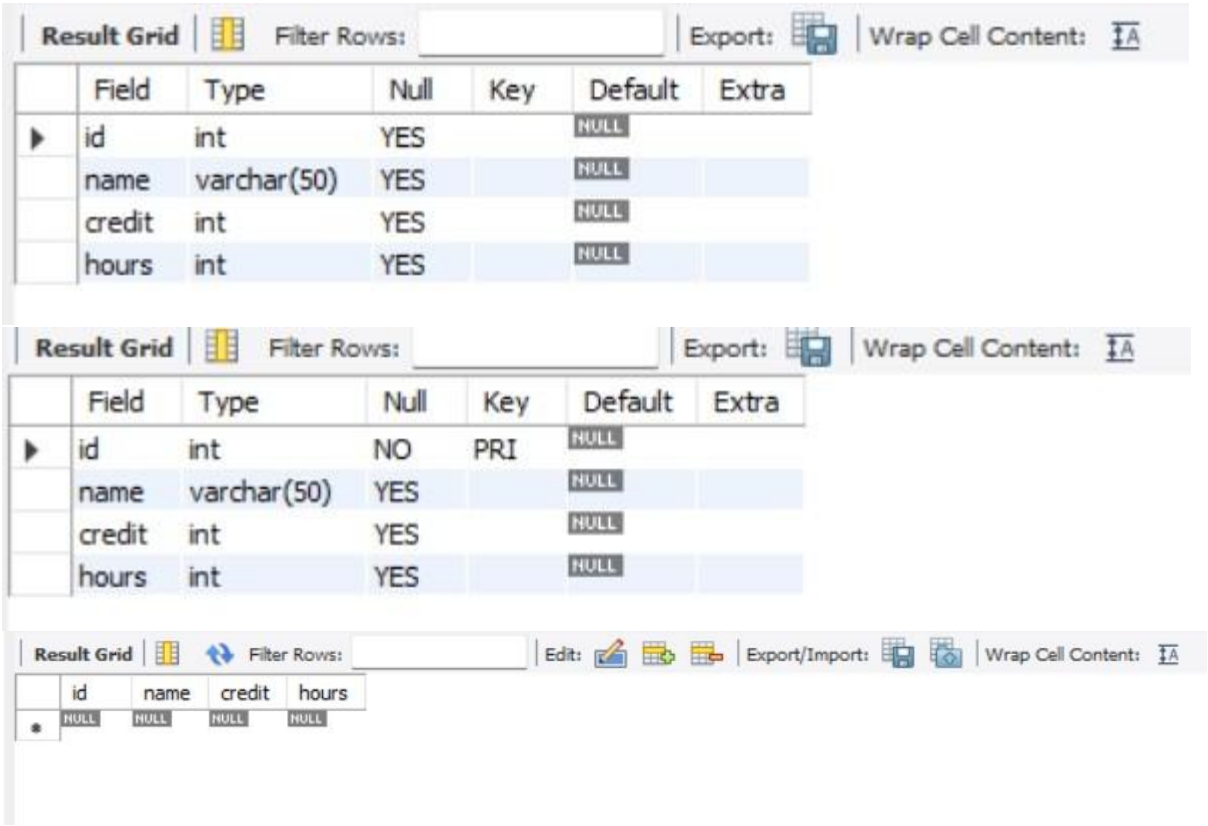
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	age	xp
1	18	NULL
3	16	NULL
5	19	NULL
6	28	NULL

6. Primary Key and Truncate

```
sql
describe h31;
alter table h31 add primary key(id);
truncate table h31; select * from
h31;
```

OUPUT



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	int	YES		NULL	
name	varchar(50)	YES		NULL	
credit	int	YES		NULL	
hours	int	YES		NULL	

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
credit	int	YES		NULL	
hours	int	YES		NULL	

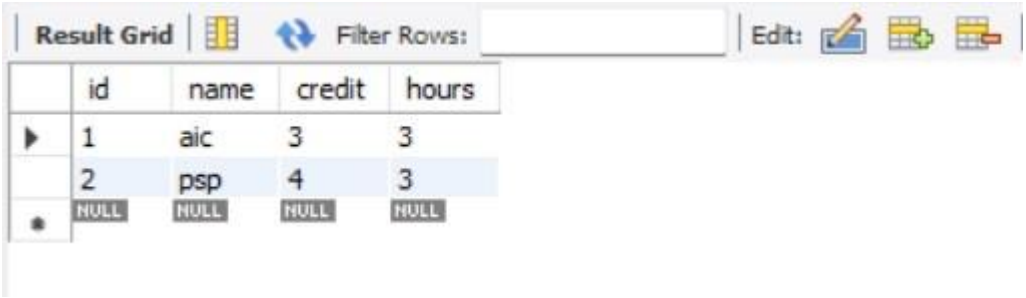
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	name	credit	hours
NULL	NULL	NULL	NULL

7. Update and Rename Operations

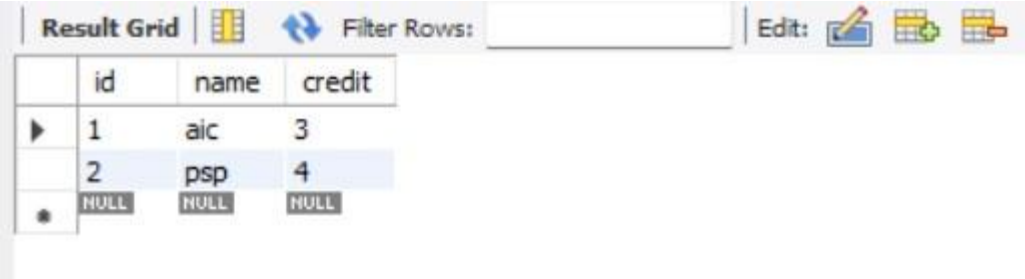
```
sql
insert into h31(id, name, credit, hours) values (1, 'aic',3,3), (2, 'psp', 4,4);
update h31 set hours=3 where id=2; select * from h31;
sql
alter table h31 rename to harsh31;
alter table harsh31 drop hours; select
* from harsh31;
```

OUPUT



Result Grid | Filter Rows: | Edit:

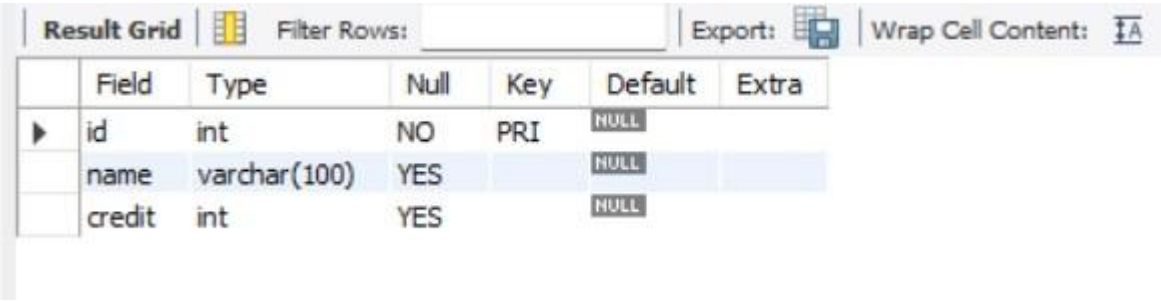
id	name	credit	hours
1	aic	3	3
2	psp	4	3
NULL	NULL	NULL	NULL



8. Final Schema Modification and Cleanup

```sql
alter table harsh31 modify name varchar(100);
describe harsh31; drop database harsh31;
```

OUPUT



PRACTICAL-3

OBJECTIVE

- Implementation and enforcement of data integrity constraints such as Primary Key, Foreign Key, Unique, and Not Null constraints.
- Writing and executing SQL queries involving aggregate functions, grouping data, and sorting results using Group By, Having, and Order By clauses.
- To design and represent the database structure using an Entity–Relationship (ER) diagram by identifying entities, attributes, relationships, and constraints for a given system.

SOFTWARE / TOOLS USED

- MySQL
- MySQL Workbench

THEORY

Data Integrity Constraints: These are rules enforced on data columns to ensure the accuracy and reliability of the data in the database.

- **Primary Key:** Uniquely identifies each record in a table.
- **Foreign Key:** A field in one table that refers to the Primary Key in another table, establishing a relationship.
- **Not Null:** Ensures that a column cannot have a NULL value.
- **Unique:** Ensures that all values in a column are different.

Aggregate Functions: Functions like SUM() and COUNT() perform a calculation on a set of values and return a single value.

Grouping and Sorting: GROUP BY is used to arrange identical data into groups, HAVING filters those groups, and ORDER BY sorts the result set.

SQL COMMANDS / PROGRAM

1. Database and Schema Definition

```
sql
drop database bank;
show databases; create
database bank; use
bank;

CREATE TABLE Bank_Detail (
```

```
    IFSC_Code VARCHAR(11) PRIMARY KEY,
    Bank_Name VARCHAR(100) NOT NULL,
    Branch_Location VARCHAR(100) NOT NULL,
    Country VARCHAR(100) DEFAULT 'INDIA'
);

CREATE TABLE Account_Holder (
    Holder_ID INT PRIMARY KEY,
    Full_Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    Phone_Number VARCHAR(15) UNIQUE
);

CREATE TABLE Account_Detail (
    Account_Number BIGINT PRIMARY KEY,
    Account_Type VARCHAR(20) NOT NULL,
    Balance DECIMAL(15, 2) DEFAULT 0.00,
    Holder_ID INT,
    IFSC_Ref VARCHAR(11),
    FOREIGN KEY (Holder_ID) REFERENCES Account_Holder(Holder_ID),
    FOREIGN KEY (IFSC_Ref) REFERENCES Bank_Detail(IFSC_Code)
);

CREATE TABLE Branch_Services (
    IFSC_Code VARCHAR(11),
    Service_Name VARCHAR(50),
    Service_Fee INT,
    PRIMARY KEY (IFSC_Code, Service_Name)
);
DESC Branch_Services;
DESC Account_Detail;
DESC Account_Holder;
DESC Bank_Detail;
```

OUTPUT

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
IFSC_Code	varchar(11)	NO	PRI	NULL	
Service_Name	varchar(50)	NO	PRI	NULL	
Service_Fee	int	YES		NULL	

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Account_Number	bigint	NO	PRI	NULL	
Account_Type	varchar(20)	NO		NULL	
Balance	decimal(15,2)	YES		0.00	
Holder_ID	int	YES	MUL	NULL	
IFSC_Ref	varchar(11)	YES	MUL	NULL	

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Holder_ID	int	NO	PRI	NULL	
Full_Name	varchar(100)	NO		NULL	
Email	varchar(100)	NO	UNI	NULL	
Phone_Number	varchar(15)	YES	UNI	NULL	

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
IFSC_Code	varchar(11)	NO	PRI	NULL	
Bank_Name	varchar(100)	NO		NULL	
Branch_Location	varchar(100)	NO		NULL	
Country	varchar(100)	YES		INDIA	

2. Data Insertion

```sql
INSERT INTO Bank_Detail (IFSC_Code, Bank_Name, Branch_Location)
```

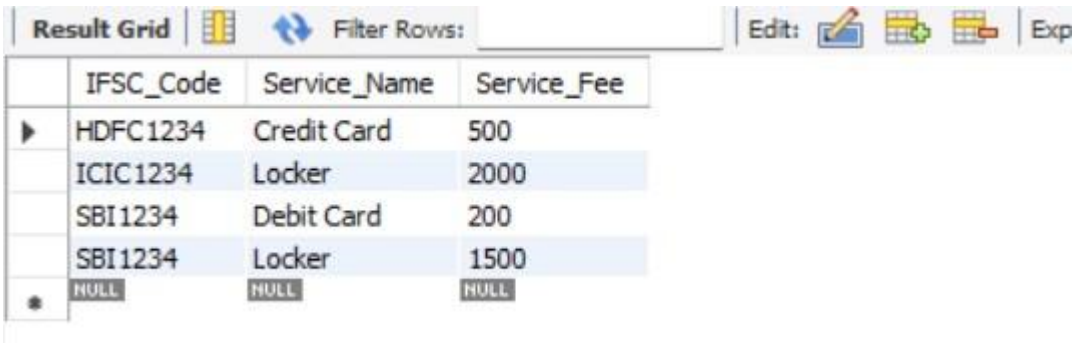
```
VALUES
  ('SBI1234', 'SBI', 'Jalandhar'),
  ('ICIC1234', 'ICICI', 'Delhi'),
  ('HDFC1234', 'HDFC', 'Chandigarh');

INSERT INTO Account_Holder (Holder_ID, Full_Name, Email, Phone_Number)
VALUES
  (1, 'Rahul', 'rahul@gmail.com', '7717413070'),
  (2, 'Rohit', 'rohit@gmail.com', '7717413071'),
  (3, 'Rudra', 'rudra@gmail.com', '7717413072');

INSERT INTO Account_Detail (Account_Number, Account_Type, Balance, Holder_ID, IFSC_Ref)
VALUES
  (1, 'Savings', 25000.00, 1, 'SBI1234'),
  (2, 'Current', 50000.00, 2, 'ICIC1234'),
  (3, 'Savings', 1200.50, 3, 'HDFC1234'),
  (4, 'Savings', 500.00, 1, 'HDFC1234');

INSERT INTO Branch_Services (IFSC_Code, Service_Name, Service_Fee)
VALUES
  ('SBI1234', 'Locker', 1500),
  ('SBI1234', 'Debit Card', 200),
  ('ICIC1234', 'Locker', 2000),
  ('HDFC1234', 'Credit Card', 500);
SELECT * FROM Branch_Services;
SELECT * FROM Account_Detail;
SELECT * FROM Account_Holder;
SELECT * FROM Bank_Detail;
```

OUTPUT



IFSC_Code	Service_Name	Service_Fee
HDFC1234	Credit Card	500
ICIC1234	Locker	2000
SBI1234	Debit Card	200
SBI1234	Locker	1500
NULL	NULL	NULL

Result Grid | Filter Rows: | Edit: | Export/Import:

Account_Number	Account_Type	Balance	Holder_ID	IFSC_Ref
1	Savings	25000.00	1	SBI1234
2	Current	50000.00	2	ICIC1234
3	Savings	1200.50	3	HDFC1234
4	Savings	500.00	1	HDFC1234
NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Edit: | Export/Import

Holder_ID	Full_Name	Email	Phone_Number
1	Rahul	rahul@gmail.com	7717413070
2	Rohit	rohit@gmail.com	7717413071
3	Rudra	rudra@gmail.com	7717413072
NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Edit: | Ex

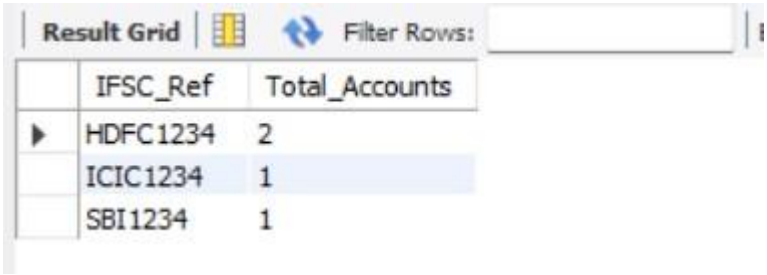
IFSC_Code	Bank_Name	Branch_Location	Country
HDFC1234	HDFC	Chandigarh	INDIA
ICIC1234	ICICI	Delhi	INDIA
SBI1234	SBI	Jalandhar	INDIA
NULL	NULL	NULL	NULL

4. Data Retrieval and Complex Queries

```
SELECT IFSC_Ref, COUNT(Account_Number) AS Total_Accounts
FROM Account_Detail
GROUP BY IFSC_Ref
ORDER BY Total_Accounts DESC;
sql
SELECT B.Bank_Name, SUM(A.Balance) AS Total_Cash
FROM Bank_Detail B
JOIN Account_Detail A ON B.IFSC_Code = A.IFSC_Ref
GROUP BY B.Bank_Name
ORDER BY Total_Cash ASC; sql
-- Having Clause: Filtering branches with wealth > 20000
```

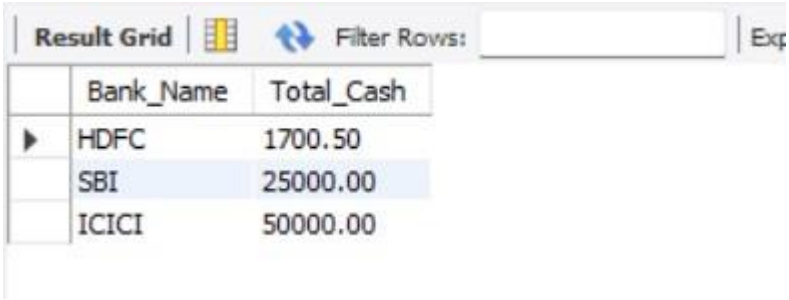
```
SELECT IFSC_Ref, SUM(Balance) AS Branch_Wealth
FROM Account_Detail
GROUP BY IFSC_Ref
HAVING SUM(Balance) > 20000
ORDER BY Branch_Wealth DESC;
```

OUTPUT



Result Grid | Filter Rows:

IFSC_Ref	Total_Accounts
HDFC1234	2
ICIC1234	1
SBI1234	1



Result Grid | Filter Rows:

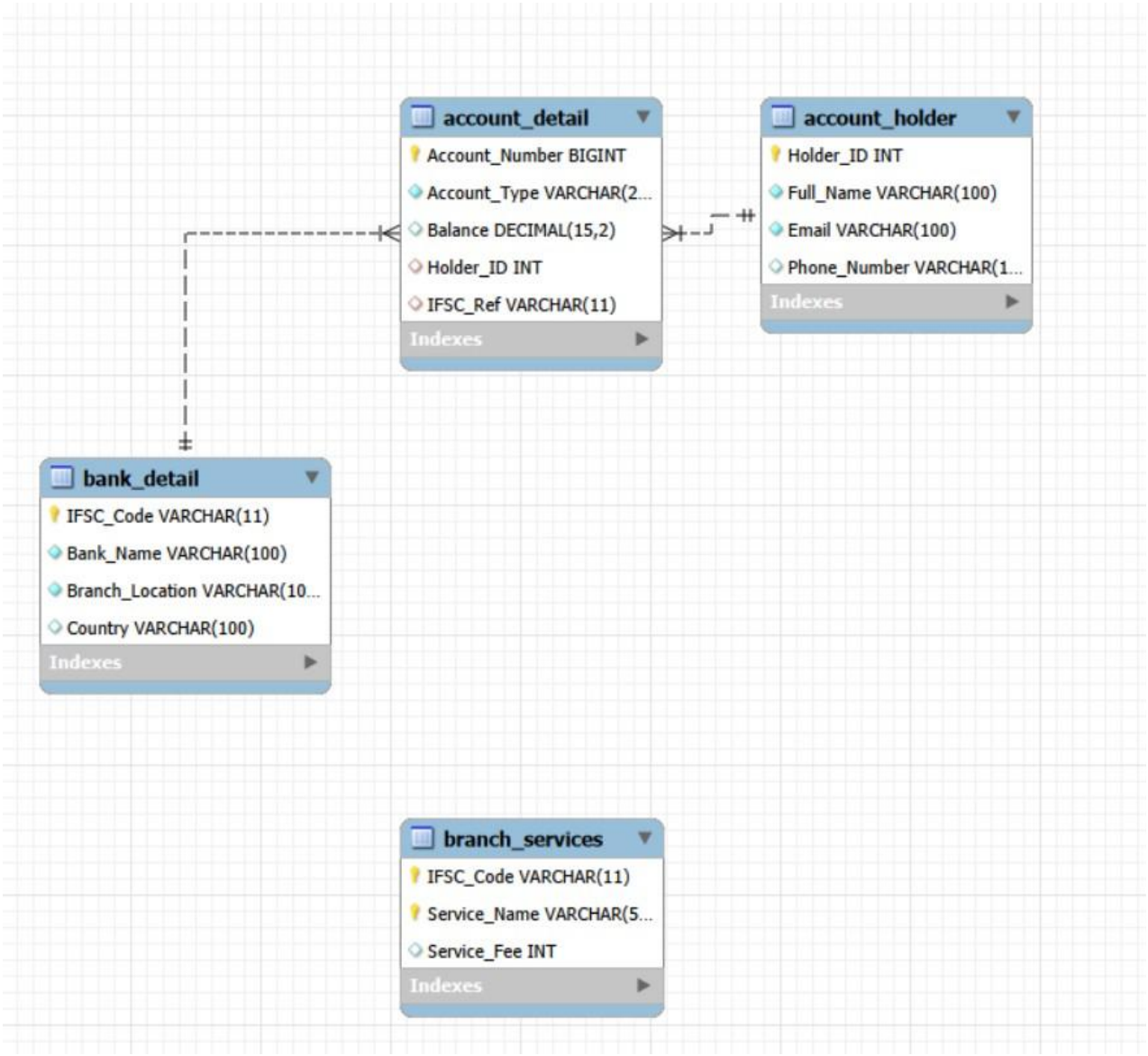
Bank_Name	Total_Cash
HDFC	1700.50
SBI	25000.00
ICICI	50000.00



Result Grid | Filter Rows: | Export:

IFSC_Ref	Branch_Wealth
ICIC1234	50000.00
SBI1234	25000.00

Entity-Relationship Representation



PRACTICAL-4

OBJECTIVE:

A. To apply relational algebra operations in SQL by writing and executing queries using UNION, INTERSECT, and MINUS operators for effective data manipulation and result combination.

B. To construct and execute SQL queries to retrieve data from multiple related tables by applying different types of joins operations.

C. To formulate and execute SQL queries containing subqueries and nested queries for performing complex data retrieval and conditional operations within the database.

Software / Tools Used

- MySQL
- MySQL Workbench

Theory

A. Set Operations (Relational Algebra)

Set operations combine the results of two or more queries **vertically** (adding rows). For these to work, both queries must have the same number of columns and compatible data types.

- **UNION:** Merges all rows from both result sets and removes any duplicates. •

INTERSECT: Returns only the rows that are present in both result sets.

- **MINUS / EXCEPT:** Returns rows from the first query that are not found in the second query. (Note: Oracle uses MINUS, while most other databases use EXCEPT).

B. Join Operations

Joins combine data **horizontally** (adding columns) from different tables based on a related column, usually a Primary Key and a Foreign Key.

- **Inner Join:** Returns only the rows where there is a match in both tables.
- **Left Join:** Returns all rows from the left table, plus any matching rows from the right table. Non-matching rows on the right show as NULL.
- **Right Join:** Returns all rows from the right table, plus any matching rows from the left table.
- **Full Join:** Returns all rows from both tables, filling in NULLs where there is no match on either side.

- **Cross Join:** Produces a Cartesian product, matching every row of the first table with every row of the second.

Subqueries and Nested Queries

A subquery is a query nested inside another statement. It allows you to use the result of one query to inform the logic of the main query.

- **Scalar Subquery:** Returns a single value (one row, one column) to be used in a comparison (like = or >).

- **Multi-Row Subquery:** Returns a list of values, used with operators like IN, ANY, or ALL.

- **Nested (Independent) Subquery:** The inner query runs once, and its result is passed to the outer query.

- **Correlated Subquery:** The inner query refers to data from the outer query and must execute once for every row processed by the outer query.

- **Derived Table:** A subquery placed in the FROM clause that acts as a temporary table for the main query.

SQL Commands / Program

```
CREATE DATABASE college_db;

USE college_db;

CREATE TABLE Department (   dept_id INT

PRIMARY KEY,   dept_name

VARCHAR(50)

);

CREATE TABLE Student (   student_id INT

PRIMARY KEY,   name VARCHAR(50),

 dept_id INT,

 FOREIGN KEY (dept_id) REFERENCES Department(dept_id)

);

CREATE TABLE Hostel (   student_id INT

PRIMARY KEY,   room_no INT,
```

```
 FOREIGN KEY (student_id) REFERENCES Student(student_id)
);
CREATE TABLE Scholarship (  student_id
INT PRIMARY KEY,  amount INT,
 FOREIGN KEY (student_id) REFERENCES Student(student_id)
);
INSERT INTO Department VALUES
```



```
(1, 'CSE'),
(2, 'ECE'),
(3, 'ME');
INSERT INTO Student VALUES
(101, 'Harsh', 1),
(102, 'Aman', 2),
(103, 'Riya', 1),
(104, 'Neha', 3),
(105, 'Karan', 2);
INSERT INTO Hostel VALUES
(101, 201),
(102, 202),
(104, 203);
INSERT INTO Scholarship VALUES
(101, 5000),
(103, 6000),
(105, 5500);
SELECT s.name, d.dept_name
FROM Student s
INNER JOIN Department d
ON s.dept_id = d.dept_id;
```

Output

	name	dept_name
▶	Harsh	CSE
	Riya	CSE
	Aman	ECE
	Karan	ECE
	Neha	ME

3

LEFT JOIN

```
SELECT st.student_id
FROM Student st
LEFT JOIN Scholarship sc
ON st.student_id = sc.student_id
WHERE sc.student_id IS NULL;
```

Output

	student_id
▶	102
	104

RIGHT JOIN

```
SELECT s.name, d.dept_name
FROM Student s
RIGHT JOIN Department d
ON s.dept_id = d.dept_id;
```

OUTPUT

	name	dept_name
▶	Harsh	CSE
	Riya	CSE
	Aman	ECE
	Karan	ECE
	Neha	ME

ECE

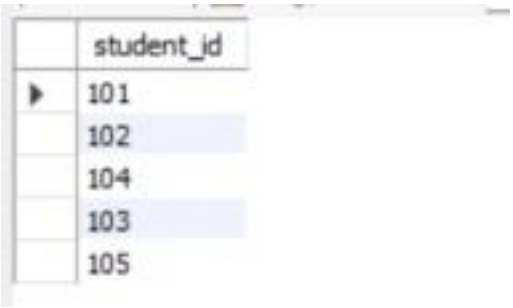
UNION

```
SELECT student_id FROM Hostel
```

UNION

SELECT student_id FROM Scholarship;

OUTPUT



	student_id
▶	101
	102
	104
	103
	105

CROSS JOIN

SELECT s.student_id, s.name, d.dept_id, d.dept_name

FROM Student s

CROSS JOIN Department d;

OUTPUT

student_id	name	dept_id	dept_name
101	Harsh	3	ME
101	Harsh	2	ECE
101	Harsh	1	CSE
102	Aman	3	ME
102	Aman	2	ECE
102	Aman	1	CSE
103	Riya	3	ME
103	Riya	2	ECE
103	Riya	1	CSE
104	Neha	3	ME
104	Neha	2	ECE
104	Neha	1	CSE
105	Karan	3	ME
105	Karan	2	ECE
105	Karan	1	CSE

NESTED COMMANDS

```
SELECT name  FROM
Student
WHERE student_id IN (
 SELECT student_id FROM Scholarship
);
```

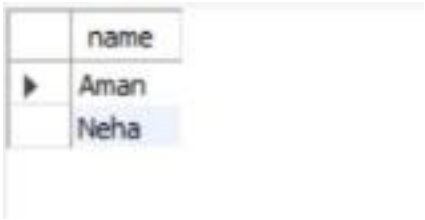
OUTPUT

name
Harsh
Riya
Karan

```
SELECT name
FROM Student
WHERE student_id NOT IN (
```

```
 SELECT student_id FROM Scholarship
);
```

OUTPUT



	name
▶	Aman
	Neha

```
SELECT name  FROM
Student
WHERE student_id IN (
 SELECT student_id
 FROM Scholarship
 WHERE amount > 5500
);
```

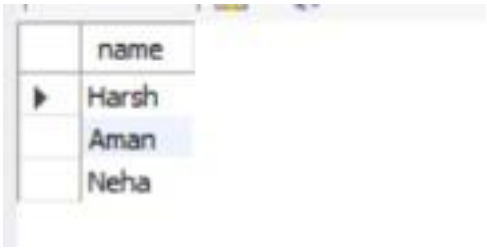
OUTPUT



	name
▶	Riya

```
SELECT name  FROM
Student
WHERE student_id IN (
 SELECT student_id FROM Hostel
);
```

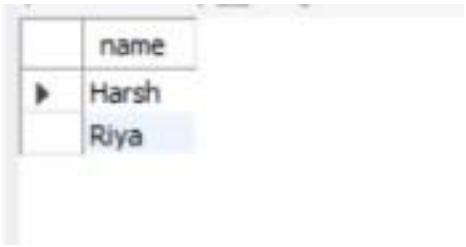

OUTPUT



	name
▶	Harsh
	Aman
	Neha

```
SELECT name  FROM
Student
WHERE dept_id = (
 SELECT dept_id
 FROM Department
 WHERE dept_name = 'CSE'
);
```

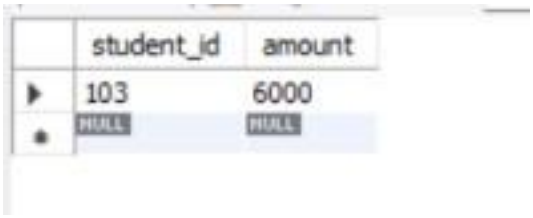
OUTPUT



	name
▶	Harsh
	Riya

```
SELECT student_id, amount
FROM Scholarship
WHERE amount = (
 SELECT MAX(amount)
 FROM Scholarship
);
```

OUTPUT



	student_id	amount
▶	103	6000
*	NULL	NULL

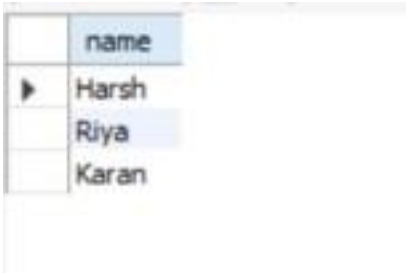
```
SELECT s.name  FROM
Student s
```



```
WHERE EXISTS (
 SELECT 1
 FROM Scholarship sc
 WHERE sc.student_id = s.student_id
```

);

OUTPUT



	name
▶	Harsh
	Riya
	Karan

```
SELECT name
FROM Student
WHERE dept_id = (
 SELECT dept_id
 FROM Department
 WHERE dept_name = 'ECE'
)
AND student_id IN (
 SELECT student_id
 FROM Scholarship
);
```

OUTPUT



	name
▶	Harsh
	Riya
	Karan

PRACTICAL-5

OBJECTIVE:

A. To utilize advanced SQL query techniques by writing and executing queries involving subqueries, set operators such as ANY, ALL, and IN, along with constraints, for performing comprehensive and accurate data analysis.

B. To create and manage database views, indexes, and sequences in order to improve data accessibility, optimize query performance, and ensure efficient data manipulation within the database system.

Software / Tools Used

- MySQL
- MySQL Workbench

Theory

A. Advanced Query Techniques

Advanced querying uses logic-based operators and constraints to filter and validate data effectively.

- Subqueries: A query nested inside another. The inner query provides a dynamic value or list that the outer query uses for its final result.
- IN Operator: Checks if a value matches any item in a specified list or subquery result.
- ANY Operator: Returns true if a comparison is true for at least one value in a set.
- ALL Operator: Returns true only if a comparison is true for every value in a set.
- Constraints: Rules (like PRIMARY KEY, UNIQUE, or NOT NULL) that prevent invalid data from being entered into the database.

B. Database Objects & Optimization

These objects are used to manage data access, speed up performance, and automate ID generation.

- Views: A "virtual table" that stores a query instead of data. It simplifies complex joins and enhances security by limiting which columns a user can see.
- Indexes: A performance tool that allows the database to find specific rows quickly without scanning the entire table (similar to a book's index).
- Sequences: An object that automatically generates a unique series of numbers, commonly used to create primary keys (like "Student ID 101, 102, 103").

SQL Commands / Program

```
CREATE DATABASE UniversitySystem;

USE UniversitySystem;

CREATE TABLE Departments (
```

```
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE Students (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    GPA DECIMAL(3,2) CHECK (GPA <= 4.0),
    DeptID INT,
    Email VARCHAR(100) UNIQUE,
    EnrollDate DATE DEFAULT (CURRENT_DATE),
    CONSTRAINT fk_dept FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
INSERT INTO Departments (DeptID, DeptName) VALUES (10, 'Computer Science');
INSERT INTO Departments (DeptID, DeptName) VALUES (20, 'Data Science');
INSERT INTO Departments (DeptID, DeptName) VALUES (30, 'Mathematics');
INSERT INTO Students (Name, GPA, DeptID, Email) VALUES ('Harsh Joshi', 3.95, 10,
'harsh@uni.edu');
INSERT INTO Students (Name, GPA, DeptID, Email) VALUES ('Karan Singh', 3.10, 10,
'karan@uni.edu');
INSERT INTO Students (Name, GPA, DeptID, Email) VALUES ('Charlie Day', 3.85, 20,
'charlie@uni.edu');
INSERT INTO Students (Name, GPA, DeptID, Email) VALUES ('Diana Prince', 2.90, 30,
'diana@uni.edu');
INSERT INTO Students (Name, GPA, DeptID, Email) VALUES ('Edward Nigma', 3.50, 20,
'edward@uni.edu');
SELECT Name, Email
FROM Students
WHERE DeptID IN (SELECT DeptID FROM Departments WHERE DeptName LIKE
'%Science%');
```

Output

	Name	Email
▶	Harsh Joshi	harsh@uni.edu
	Karan Singh	karan@uni.edu
	Charlie Day	charlie@uni.edu
	Edward Nigma	edward@uni.edu

ANY

SELECT Name, GPA FROM Students

WHERE GPA > ANY (SELECT GPA FROM Students WHERE DeptID = 30);

Output

	Name	GPA
▶	Harsh Joshi	3.95
	Karan Singh	3.10
	Charlie Day	3.85
	Edward Nigma	3.50

ALL

SELECT Name, GPA FROM Students

WHERE GPA > ALL (SELECT GPA FROM Students WHERE DeptID = 30); **Output**

	Name	GPA
▶	Harsh Joshi	3.95
	Karan Singh	3.10
	Charlie Day	3.85
	Edward Nigma	3.50

VIEW

CREATE VIEW HonorRoll AS

SELECT Name, GPA

FROM Students

WHERE GPA >= 3.5;

SELECT * FROM HonorRoll;

Output

	Name	GPA
▶	Harsh Joshi	3.95
	Charlie Day	3.85
	Edward Nigma	3.50

INDEX

CREATE INDEX idx_student_name ON Students(Name);

SELECT StudentID AS ID FROM Students

UNION

SELECT DeptID FROM Departments;

Output

	ID
▶	1
	2
	3
	5
	4
	10
	20
	30

NOT IN

SELECT DeptID FROM Departments

WHERE DeptID NOT IN (SELECT DISTINCT DeptID FROM Students WHERE DeptID IS NOT NULL);

Output

	DeptID
*	NULL

PRACTICAL NO-6

OBJECTIVE: Data Insertion and Execution

Software / Tools Used:

- MySQL
- MySQL workbench

SQL Commands

Q1. Table Creation – University Database

Create the following tables in MySQL with appropriate constraints:

1. Student

o student_no (PK), ssn (UNIQUE), first_name, last_name, birth_date, sex, class, major_dept, minor_dept, degree_program, current_address, current_phone, permanent_address, city, state, zip

2. Department

o dept_code (PK), dept_name (UNIQUE), office_no, office_phone, college

3. Course

o course_no (PK), course_name, description, semester_hours, level, dept_code (FK)

4. Section

o section_id (PK), course_no (FK), instructor, semester, year, section_no

5. Grade_Report

o student_no (FK), section_id (FK), letter_grade, numeric_grade (0–4), composite PK (student_no, section_id)

SQL

```
create database university;
use university;
CREATE TABLE Department (
   dept_code VARCHAR(10) PRIMARY KEY,
   dept_name VARCHAR(100) UNIQUE,
   office_no VARCHAR(20),
   office_phone VARCHAR(15),
   college VARCHAR(100)
);

CREATE TABLE Student (
   student_no INT PRIMARY KEY,
   ssn VARCHAR(20) UNIQUE,
   first_name VARCHAR(50),
   last_name VARCHAR(50),
   birth_date DATE,
```

```
    sex CHAR(1),
    class VARCHAR(20),
    major_dept VARCHAR(10),
    minor_dept VARCHAR(10),
    degree_program VARCHAR(50),
    current_address VARCHAR(200),
    current_phone VARCHAR(15),
    permanent_address VARCHAR(200),
    city VARCHAR(50),
    state VARCHAR(50),
    zip VARCHAR(10),
    FOREIGN KEY (major_dept) REFERENCES Department(dept_code),
    FOREIGN KEY (minor_dept) REFERENCES Department(dept_code)
);

CREATE TABLE Course (
    course_no VARCHAR(10) PRIMARY KEY,
    course_name VARCHAR(100),
    description TEXT,
    semester_hours INT,
    level VARCHAR(20),
    dept_code VARCHAR(10),
    FOREIGN KEY (dept_code) REFERENCES Department(dept_code)
);

CREATE TABLE Section (
    section_id INT PRIMARY KEY,
    course_no VARCHAR(10),
    instructor VARCHAR(100),
    semester VARCHAR(20),
    year INT,
    section_no INT,
    FOREIGN KEY (course_no) REFERENCES Course(course_no)
);

CREATE TABLE Grade_Report (
    student_no INT,
    section_id INT,
    letter_grade CHAR(2),
    numeric_grade DECIMAL(2,1),
    PRIMARY KEY (student_no, section_id),
    FOREIGN KEY (student_no) REFERENCES Student(student_no),
    FOREIGN KEY (section_id) REFERENCES Section(section_id),
    CHECK (numeric_grade BETWEEN 0 AND 4)
```

);
drop database university;

Q2. Table Creation – Company Database
Create the following tables:
 employee (employee_name PK, street, city)
 works (employee_name FK, company_name, salary)
 company (company_name PK, city)
 manages (employee_name FK, manager_name)

SQL

```
create database company;
use company;
CREATE TABLE employee (
    employee_name VARCHAR(50) PRIMARY KEY,
    street VARCHAR(100),
    city VARCHAR(50)
);
CREATE TABLE company (
    company_name VARCHAR(50) PRIMARY KEY,
    city VARCHAR(50)
);
CREATE TABLE works (
    employee_name VARCHAR(50),
    company_name VARCHAR(50),
    salary INT,
    FOREIGN KEY (employee_name) REFERENCES employee(employee_name),
    FOREIGN KEY (company_name) REFERENCES company(company_name)
);
CREATE TABLE manages (
    employee_name VARCHAR(50),
    manager_name VARCHAR(50),
    FOREIGN KEY (employee_name) REFERENCES employee(employee_name)
);
INSERT INTO employee VALUES
('Amit', 'Street1', 'Delhi'),
('Ravi', 'Street2', 'Mumbai'),
('Neha', 'Street3', 'Delhi'),
('Sita', 'Street4', 'Chennai'),
('Karan', 'Street5', 'Mumbai'),
('Priya', 'Street6', 'Delhi'),
('Arjun', 'Street7', 'Pune'),
('Meena', 'Street8', 'Kolkata'),
```

('Rahul', 'Street9', 'Delhi'),
('Anjali', 'Street10', 'Bangalore');
INSERT INTO company VALUES
('First Bank Corporation', 'Delhi'),
('Small Bank Corporation', 'Mumbai'),
('TechCorp', 'Chennai'),
('FinanceHub', 'Delhi'),
('GlobalSoft', 'Bangalore'),
('NextGen', 'Pune'),
('InfoTech', 'Kolkata'),
('SecureBank', 'Mumbai'),
('DataWorks', 'Delhi'),
('AlphaSolutions', 'Chennai');
INSERT INTO works VALUES
('Amit', 'First Bank Corporation', 12000),
('Ravi', 'First Bank Corporation', 9000),
('Neha', 'Small Bank Corporation', 8000),
('Sita', 'TechCorp', 15000),
('Karan', 'FinanceHub', 11000),
('Priya', 'First Bank Corporation', 13000),
('Arjun', 'NextGen', 14000),
('Meena', 'InfoTech', 7000),
('Rahul', 'Small Bank Corporation', 8500),
('Anjali', 'GlobalSoft', 16000);
INSERT INTO manages VALUES
('Amit', 'Ravi'),
('Neha', 'Amit'),
('Sita', 'Karan'),
('Priya', 'Amit'),
('Arjun', 'Sita'),
('Meena', 'Neha'),
('Rahul', 'Karan'),
('Anjali', 'Priya'),
('Karan', 'Ravi'),
('Ravi', 'Amit');

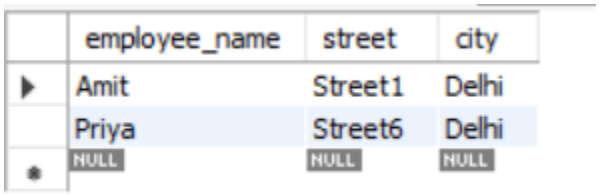
Q3. SQL Queries – Company Database
Write SQL queries for the following:
a) Retrieve employee name, street, and city for employees working in “First Bank Corporation” with salary greater than 10000.
b) Retrieve names of employees who do not work for “First Bank Corporation”.
c) Retrieve employees who earn more than every employee of “Small Bank Corporation”.
d) Retrieve companies located in all cities where “Small Bank Corporation” operates.

e) Retrieve the company with the maximum number of employees.
f) Retrieve companies whose average salary is greater than the average salary of “First Bank Corporation”.

SQL

```
select employee_name, street, city
from employee
where employee_name IN
(select employee_name
from works
where company_name="First Bank Corporation" AND salary>10000);
```

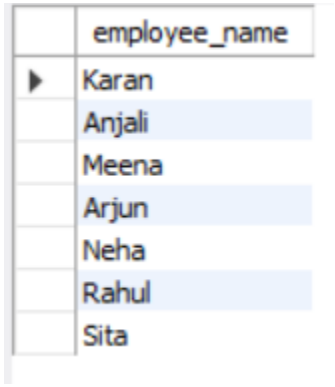
OUTPUT



employee_name	street	city
Amit	Street1	Delhi
Priya	Street6	Delhi
NULL	NULL	NULL

```
select employee_name
from works
where company_name<>"First Bank Corporation";
```

OUTPUT



employee_name
Karan
Anjali
Meena
Arjun
Neha
Rahul
Sita

```
SELECT employee_name
FROM works
WHERE salary > ALL (
   SELECT salary
   FROM works
   WHERE company_name = 'Small Bank Corporation'
);
```

OUTPUT

employee_name
Amit
Ravi
Sita
Karan
Priya
Arjun
Anjali

```
SELECT company_name
FROM works
GROUP BY company_name
ORDER BY COUNT(*) DESC
LIMIT 1;
```

OUTPUT

company_name
First Bank Corporation

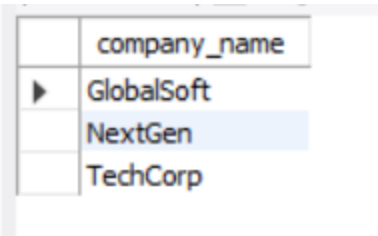
```
SELECT company_name
FROM company
WHERE city IN (
   SELECT city
   FROM company
   WHERE company_name = 'Small Bank Corporation'
);
```

OUTPUT

company_name
SecureBank
Small Bank Corporation
NULL

```
SELECT company_name
FROM works
GROUP BY company_name
HAVING AVG(salary) > (
   SELECT AVG(salary)
   FROM works
   WHERE company_name = 'First Bank Corporation'
);
```

OUTPUT



company_name
GlobalSoft
NextGen
TechCorp

Q4. Table Creation – Bank Database
Create the following tables with constraints:
 branch (branch_name PK, branch_city, assets)
 customer (customer_name PK, customer_street, customer_city)

 loan (loan_number PK, branch_name FK, amount)
 borrower (customer_name FK, loan_number FK, composite PK)
 account (account_number PK, branch_name FK, balance)
 depositor (customer_name FK, account_number FK, composite PK)
Q5. SQL Queries – Bank Database
Write SQL queries for the following:
a) Retrieve all customers who have an account but no loan.
b) Retrieve customers who live on the same street and city as “Smith”.
c) Retrieve names of branches where customers have accounts and live in “Harrison”.
d) Retrieve customers who have accounts in all branches located in “Brooklyn”.
e) Retrieve the total sum of all loan amounts.
f) Retrieve branches with assets greater than at least one branch in “Brooklyn”.

SQL

```
create database bank1;
use bank1;
CREATE TABLE branch (
    branch_name VARCHAR(50) PRIMARY KEY,
    branch_city VARCHAR(50),
    assets INT
);

CREATE TABLE customer (
    customer_name VARCHAR(50) PRIMARY KEY,
    customer_street VARCHAR(100),
    customer_city VARCHAR(50)
);

CREATE TABLE loan (
    loan_number INT PRIMARY KEY,
    branch_name VARCHAR(50),
```

```
    amount INT,
    FOREIGN KEY (branch_name) REFERENCES branch(branch_name)
);

CREATE TABLE borrower (
    customer_name VARCHAR(50),
    loan_number INT,
    PRIMARY KEY (customer_name, loan_number),
    FOREIGN KEY (customer_name) REFERENCES customer(customer_name),
    FOREIGN KEY (loan_number) REFERENCES loan(loan_number)
);

CREATE TABLE account (
    account_number INT PRIMARY KEY,
    branch_name VARCHAR(50),
    balance INT,
    FOREIGN KEY (branch_name) REFERENCES branch(branch_name)
);

CREATE TABLE depositor (
    customer_name VARCHAR(50),
    account_number INT,
    PRIMARY KEY (customer_name, account_number),
    FOREIGN KEY (customer_name) REFERENCES customer(customer_name),
    FOREIGN KEY (account_number) REFERENCES account(account_number)
);

INSERT INTO branch VALUES
('B1','Brooklyn',500000),
('B2','Brooklyn',300000),
('B3','Harrison',400000),
('B4','Harrison',450000),
('B5','Delhi',600000),
('B6','Delhi',650000),
('B7','Mumbai',350000),
('B8','Mumbai',370000),
('B9','Chennai',450000),
('B10','Pune',250000);

INSERT INTO customer VALUES
('Smith','Street1','Harrison'),
('John','Street1','Harrison'),
('Amit','Street2','Delhi'),
('Priya','Street2','Delhi'),
```

('Ravi','Street3','Mumbai'),
('Karan','Street3','Mumbai'),
('Neha','Street4','Harrison'),
('Sita','Street5','Chennai'),
('Arjun','Street6','Brooklyn'),
('Rahul','Street7','Brooklyn');

INSERT INTO account VALUES
(201,'B1',5000),
(202,'B2',6000),
(203,'B3',7000),
(204,'B4',8000),
(205,'B5',9000),
(206,'B6',7500),
(207,'B1',8500),
(208,'B2',4000),
(209,'B3',9500),
(210,'B4',3000);

INSERT INTO depositor VALUES
('Smith',201),
('Smith',202),
('John',201),
('John',202),
('Amit',203),
('Priya',204),
('Ravi',205),
('Karan',206),
('Neha',207),
('Sita',208);

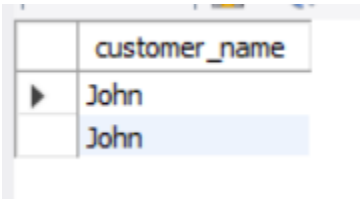
INSERT INTO loan VALUES
(101,'B1',10000),
(102,'B2',20000),
(103,'B3',15000),
(104,'B4',30000),
(105,'B5',25000),
(106,'B6',18000),
(107,'B7',22000),
(108,'B8',12000),
(109,'B9',35000),
(110,'B10',9000);

INSERT INTO borrower VALUES

('Smith',101),
('Amit',102),
('Ravi',103),
('Neha',104),
('Sita',105),
('Karan',106),
('Priya',107);

```
SELECT customer_name
FROM depositor
WHERE customer_name NOT IN (
   SELECT customer_name FROM borrower
);
```

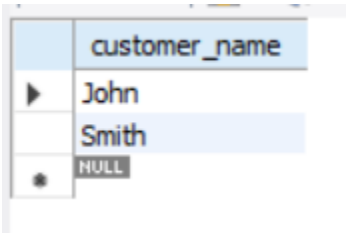
OUTPUT



customer_name
John
John

```
SELECT customer_name
FROM customer
WHERE (customer_street, customer_city) = (
   SELECT customer_street, customer_city
   FROM customer
   WHERE customer_name = 'Smith'
);
```

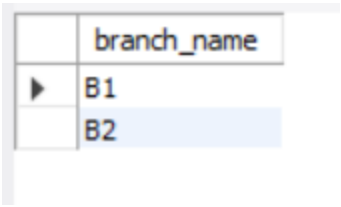
OUTPUT



customer_name
John
Smith
NULL

```
SELECT DISTINCT b.branch_name
FROM branch b
JOIN account a ON b.branch_name = a.branch_name
JOIN depositor d ON a.account_number = d.account_number
JOIN customer c ON d.customer_name = c.customer_name
WHERE c.customer_city = 'Harrison';
```

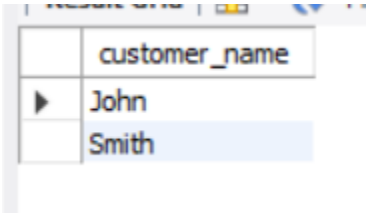
OUTPUT



branch_name
B1
B2

```
SELECT d.customer_name
FROM depositor d
JOIN account a ON d.account_number = a.account_number
GROUP BY d.customer_name
HAVING COUNT(DISTINCT a.branch_name) = (
   SELECT COUNT(*)
   FROM branch
   WHERE branch_city = 'Brooklyn'
);
```

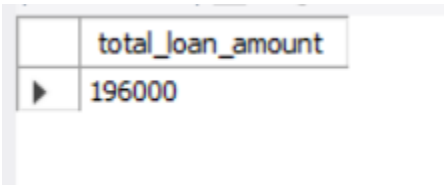
OUTPUT



customer_name
John
Smith

```
SELECT SUM(amount) AS total_loan_amount
FROM loan;
```

OUTPUT



total_loan_amount
196000

```
SELECT branch_name
FROM branch
WHERE assets > ANY (
   SELECT assets
   FROM branch
   WHERE branch_city = 'Brooklyn'
);
```

OUTPUT

branch_name
B1
B3
B4
B5
B6
B7
B8
B9
NULL

Q7. Advanced Tasks
a) Create a VIEW for employees working in First Bank Corporation.
b) Create an INDEX on salary in the works table.
c) Write a query using GROUP BY and HAVING.

SQL

```
CREATE VIEW first_bank_employees AS
SELECT e.employee_name, e.street, e.city, w.salary
FROM employee e
JOIN works w ON e.employee_name = w.employee_name
WHERE w.company_name = 'First Bank Corporation';
```

OUTPUT

company_name
SecureBank
Small Bank Corporation
NULL

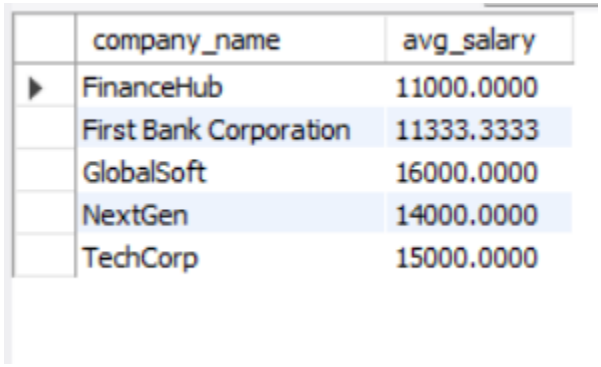
```
CREATE INDEX idx_salary
ON works(salary);
```

OUTPUT

company_name
GlobalSoft
NextGen
TechCorp

```
SELECT company_name, AVG(salary) AS avg_salary
FROM works
GROUP BY company_name
HAVING AVG(salary) > 10000;
```

OUTPUT



company_name	avg_salary
FinanceHub	11000.0000
First Bank Corporation	11333.3333
GlobalSoft	16000.0000
NextGen	14000.0000
TechCorp	15000.0000

PRACTICAL NO-6

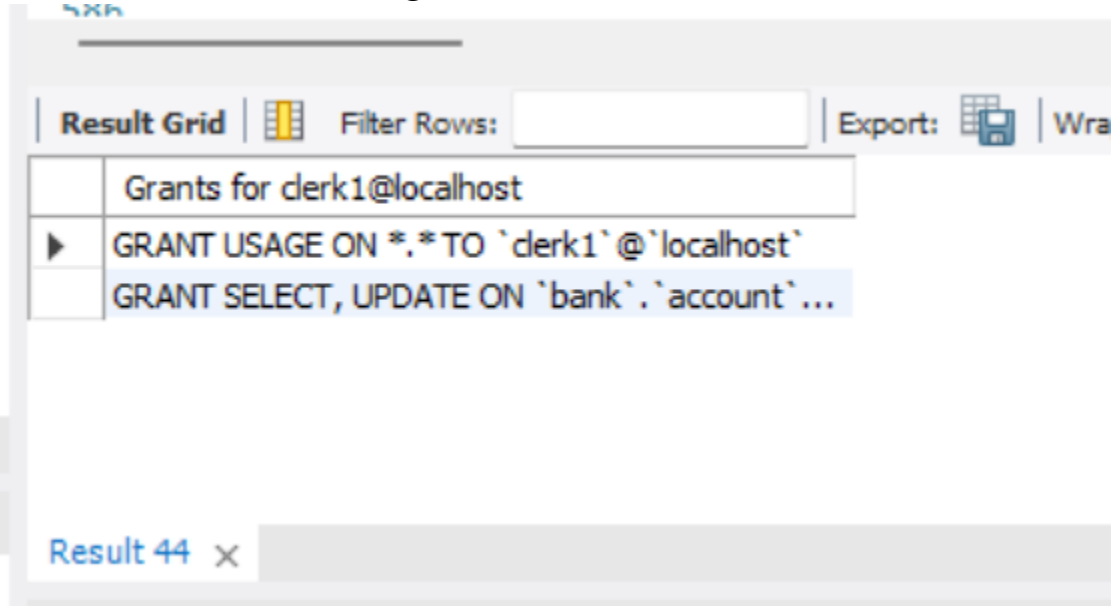
OBJECTIVE: Implement Data Control Language (DCL) and Transaction Control Language (TCL) commands for managing database security and transactions.

Software / Tools Used:

• MySQL
• MySQL workbench

Example 1
You are managing a bank database.
Situation:
• Table: Accounts(account_id, name, balance)
• User: clerk1
Tasks:
a. Give clerk1 permission to view and update account details.

```
use Bank;
CREATE USER 'clerk1'@'localhost' IDENTIFIED BY '1234';
GRANT SELECT, UPDATE
ON account
TO 'clerk1'@'localhost';
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'clerk1'@'localhost';
```



b. Transfer ₹5000 from account A to account B.
c. If any error occurs, undo the transaction.
d. If everything is correct, save changes permanently.

```
START TRANSACTION;

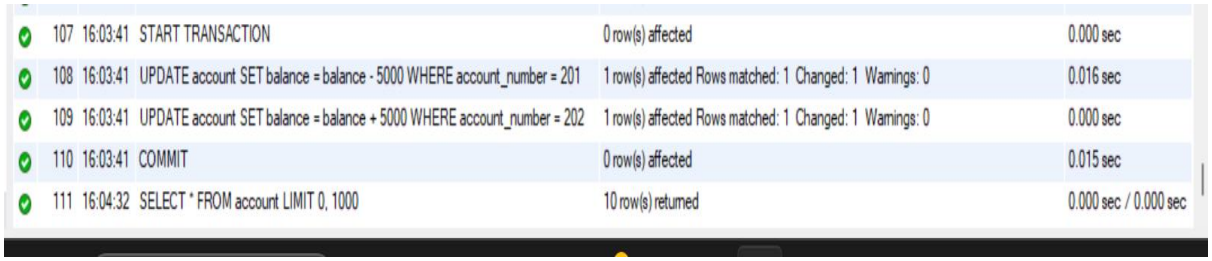
-- Deduct ₹5000 from Account A
UPDATE account
```

```
SET balance = balance - 5000
WHERE account_number = 201;   -- (Account A)

-- Add ₹5000 to Account B
UPDATE account
SET balance = balance + 5000
WHERE account_number = 202;   -- (Account B)

-- If everything is correct
COMMIT;

-- If error occurs (use this instead of commit)
-- ROLLBACK;
```



107	16:03:41	START TRANSACTION	0 row(s) affected	0.000 sec
108	16:03:41	UPDATE account SET balance = balance - 5000 WHERE account_number = 201	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.016 sec
109	16:03:41	UPDATE account SET balance = balance + 5000 WHERE account_number = 202	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
110	16:03:41	COMMIT	0 row(s) affected	0.015 sec
111	16:04:32	SELECT * FROM account LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

Example 2

You are both DBA + Developer.

Situation:

• Table: Projects(id, name, budget)

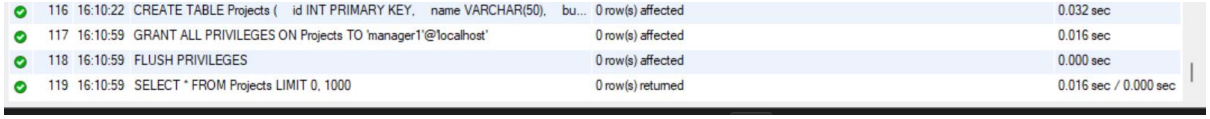
• User: manager1

Tasks:

a. Grant ALL privileges to manager1.

Code:

```
CREATE TABLE Projects (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    budget INT
);
GRANT ALL PRIVILEGES
ON Projects
TO 'manager1'@'localhost';
FLUSH PRIVILEGES;
SELECT * FROM Projects;
```



116	16:10:22	CREATE TABLE Projects (id INT PRIMARY KEY, name VARCHAR(50), bu...	0 row(s) affected	0.032 sec
117	16:10:59	GRANT ALL PRIVILEGES ON Projects TO 'manager1'@'localhost'	0 row(s) affected	0.016 sec
118	16:10:59	FLUSH PRIVILEGES	0 row(s) affected	0.000 sec
119	16:10:59	SELECT * FROM Projects LIMIT 0, 1000	0 row(s) returned	0.016 sec / 0.000 sec

b. Insert 2 records.

c. Create SAVEPOINT after first insert.